

AdaLightLog: Enhancing Application Logs Anomaly Detection via Adaptive Federating Learning

Danilo Menegatti, Emanuele De Santis, Stefano Felli, and Alessandro Giuseppe

Department of Computer, Control and Management Engineering ‘Antonio Ruberti’,
Sapienza University of Rome, Rome, Italy

Abstract. The role of anomaly detection systems in Critical Infrastructures (CIs) is critical due to the complexity of CIs and their control systems, which are usually implemented by computer-based controllers that constantly produce logs of their activities. Moreover, many CIs, located in different locations or even belonging to different companies, may share similar application software for controlling the CIs themselves. The goal of this work is to use such logs to perform automatic anomaly detection in a federated learning (FL) paradigm, which ensures that no data is exchanged between sites to train the anomaly detection models, but each learning agent learns on its own data, leveraging the knowledge acquired by the other agents. Our proposed approach - ADALIGHTLOG - which implements a modified FL paradigm with adaptive loss functions at local servers side and weighted averaging of local server models, so to differentiate the quality of the different local servers’ models in the global averaging, is tested against state-of-the-art methods and shows an improvement in performance in terms of accuracy, precision and recall with respect to the standard FL implementation (FEDAVG). Furthermore, a comparison between different metrics for the adaptive loss functions and the dynamic weights is presented.

Keywords: Federated Learning · Log Anomaly Detection · Adaptive Loss · Weighted Averaging.

1 Introduction

Anomaly detection plays an important role in the framework of critical infrastructures (CIs), that is any system which is essential for providing vital economic and social functions [5]. In general, their architecture is composed of two main components, that is Industrial Control Systems (ICSs) and innovative technologies such as Internet of Things (IoT) [36] which can be seen as the means to achieve the efficient control and automation of industrial processes.

Although such mixture proved to be effective in multiple applications [20, 37], it makes the overall framework inevitably susceptible to cyber-physical attacks, since the ICSs, which interact directly with the physical instruments and devices, also find themselves interacting with the outside environment. One of

the possible strategies which proved to be effective in mitigating the risk of an attack, if not preventing it, is anomaly detection; having the frequency of malicious attacks surged in the last years due to CIs' expanded connectivity, the aforementioned techniques provide a useful tool in highlight anomalies from the expected behaviour, hence leading the way to appropriate and preventive counter-measures.

This work proposes a novel Federated Learning -based distributed anomaly detection system that analyzes application logs to detect possible anomalies at software level. The proposed algorithm makes use of Federated Learning (FL) methodology to preserve privacy among the federation. This is justified by the fact that application logs may contain sensitive and/or confidential information, that the members of the federation do not want to share for the learning purpose. Instead, the proposed approach, as in the most common FL approaches, do not share any data among the learning agents, but only the knowledge they acquired during training phases.

Moreover, application logs are usually expressed in some human-readable natural language, thus proper log parsing (based on the intuition of [6] and [45], that application logs usually follows a limited set of different templates) and embedding techniques are put in place.

In addition, this work proposes to apply weighted loss functions at learning agents level based on validation metrics, so to improve the performance of the overall FL training process. In this regard, an analysis of the performance of the presented anomaly detection system against the various validation metrics used for the custom loss functions has been performed, so to validate the overall approach.

The main highlights of the proposed approach are as follows:

- Application of an adaptive FL methodology to the log anomaly detection problem
- Introduction of a performance-based weighted averaging procedure to account for data distribution disparity and resilience of the federation
- Improvement of the training procedure via the introduction of an adaptive loss function linked to validation metrics
- Metric-based comparative analysis of the training performance

The remainder of the paper is organized as follows: Section 2 describes the works in the literature that are relevant to the log anomaly detection problem and its decentralisation; Section 3 formulates the problem and details the proposed approach; Section 4 presents the simulations results; Section 5 draws the conclusion and discussion future research directions.

2 Related Works

Although several automatic anomaly detectors have been developed during the past few years due to the massive amount of logs generated on a daily basis,

some issues may arise when sensitive information about the operation of CIs is included in these data, preventing them to be shared for training purposes.

Anomaly detection in its broader sense has been the focus of many works in the last years, given the importance to gain situational awareness of the state of any CI, by giving the possibility to promptly apply adequate countermeasures in case of attacks or other anomalies. In particular, works like [43, 19] make use of purely statistical methods for detecting anomalies, by exploiting statistical properties of the attack patterns. These methods, while are very suited for specific applications and attacks, may be hard to generalize to novel attacks and more complex CIs.

Machine learning methodologies started then to increase their success in the literature, given the possibility to rely much less on specific knowledge of the attack types and the CI details. Works like [1, 15, 18] started to use methodologies like Support Vector Machines (SVM) and Bayesian Networks to classify anomalies on computer systems. Such methodologies have been outperformed by the advances of Deep Learning (DL) models, like in [12, 32], which specialized on finding anomalies in computer networks.

As for application logs anomaly detection, other recent works are available in the literature. In particular, [14] performed anomaly detection by parsing and embedding logs as in our approach, evaluating the performances of a set of supervised and unsupervised learning techniques. This topic has been also studied by [6], which proposes a complete framework for anomaly detection of application logs, by exploiting the same parsing and embedding of the previous work, but proposing a Temporal Convolutional Network (TCN) for log key anomaly detection and a parameter-based anomaly detection of the parameters in the log keys which are considered to be normal in the first step. A lighter version of [6] is provided by [45], which does not include parameter-based anomaly detection and simplifies the parsing and embedding. The main differences between these works and our proposed work is that we apply a novel FL framework preserving logs privacy among the clients of the federation, while in the aforementioned works all the data are considered to stay on a single place.

As mentioned before, this paper leverages on the concept of FL which has emerged as a promising approach for the distributed learning of a DL model. Firstly introduced in [28] alongside the FEDERATED AVERAGING (FEDAVG) algorithm, it found application very rapidly in multiple fields, for instance within IoT and eHealth scenarios [31, 2], due to its communication efficient privacy-preserving features which are associated with the complete lack of data exchange. A federation of clients learns a shared DL model, that is a neural network (NN), via an iterative procedure carried out by a server updating it by means of the weighted average of the models sent by the clients after training them over their own local data (*communication round*).

The main characteristic of the FL paradigm lies in its ability to deal with data *as-it-is*, even if non-independently and identically distributed (non-IID), without the need to collect it in a centralised fashion nor re-distribute it. Moreover, it may happen that clients could differ in terms of computational capabilities,

thus performing a variable amount of local work (*stragglers*), which leads to the delay of the overall learning. Multiple strategies have been developed to mitigate the effect of such system heterogeneity; for instance, in [44] a stopping-and-weight balancing (SAWB) method is proposed with the aim of taking into account stragglers’ partial model updates across the non-straggler clients, while the authors of [23] directly take into account variable computational capabilities of the federation by adding a proximal term to the loss function that each client minimizes during local training.

Over the past few years, several works have attempted to improve the original FL formulation reducing the amount of data being exchanged between the server and the clients [41, 39], optimizing the loss function to achieve improved performance [9, 23], and introducing privacy-preserving safeguards such as secure multiparty computation [3] and differential privacy [8].

While FEDAVG was first applied within the context of next-word prediction in [11], its $\mathcal{O}(\frac{1}{T})$ convergence rate was demonstrated later on by the authors of [24], with T being the number of local training iterations at clients’ level. The topic of convergence is also explored in [34], where a FEDAVG variant, based on the inclusion of Nesterov momentum [4], is proved to achieve a faster convergence rate in convex settings.

As mentioned before, this work envisages the use of the FL paradigm for anomaly detection purposes. To this end, [33] was one of the first works to leverage on the FL setting to train an intrusion detection model over the CIC-IDS2017 dataset. Building on top of that, the authors of [38] propose FEDSAM, a FEDAVG-like algorithm which introduces the concept of sampling size, namely the amount of data used for local training in each communication round, to deal with the presence of stragglers, and a distributed min-max scaling strategy to deal with data heterogeneity. The problem of data distribution is addressed also in [30], where clients of the federation are aggregated on the basis of the statistical properties of their local dataset and FEDAVG runs over the resulting clusters. Privacy concerns are addressed in [40] and [16]; while the former proposes a hierarchical approach together with blockchain technology to validate the model updates performed by the server, the latter envisages the use of SSL/TLS protocol.

This work aims to solve the problem of anomaly detection starting from the logs produced at application level while guaranteeing the anonymity of the collected user data leveraging on the ADAFED algorithm [9] conversely from [22] and [10] which rely on FEDAVG. In particular, we differ from the former in terms of robustness to malicious attacks by the inclusion of a model testing procedure before its update at the server level, instead of the simple addition of Gaussian noise to the clients’ model parameters, and from the latter since the overall computational burden of the learning process is optimized by the introduction of an adaptive loss function instead of relying on the lottery ticket hypothesis [7] to find sparse sub-networks within the entire federation, making the corresponding clients more prone to malicious attacks.

3 Distributed Anomaly Detection

In this work, the problem of detecting anomalies on distributed log data is formulated as a binary classification problem, which envisages to decide whether each instance of a log represents a normal or abnormal activity.

Differently from standard anomaly identification procedure, we do not collect raw logs in a centralised fashion and then perform a uniform pre-processing procedure, instead we deal with scenarios where logs are stored on the source device, which is in charge of their parsing, sampling, and embedding. Fig. 1 illustrates the proposed workflow.

The resulting distributed classification problem is solved via an adaptive FL procedure, emphasizing the collaboration among different devices.

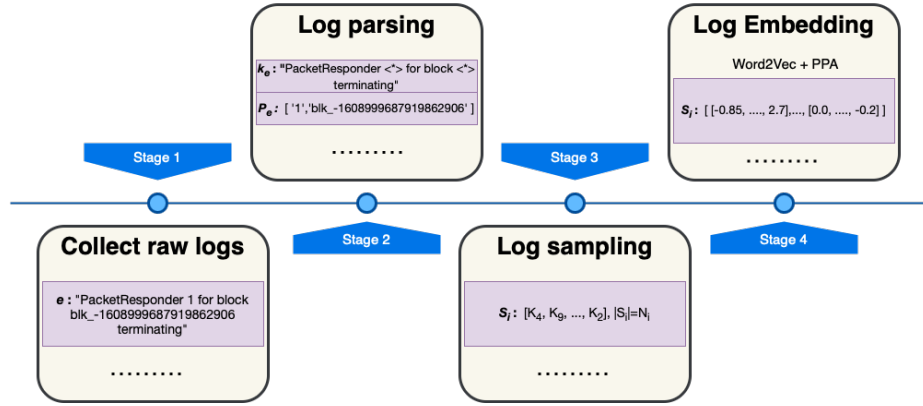


Fig. 1: Chosen log processing framework.

3.1 Log Processing Workflow

The pre-processing stage of logs is aimed at transforming raw logs into meaningful representations for the training of the learning model. We encompass the various strategies proposed in [45] which enable real-time log mapping with minimal computational resource consumption, that is processing logs in sequential phases, as depicted in Fig. 1.

Considering their entries as textual data containing various fields, such as timestamp and details regarding system processing, it is crucial to eliminate unnecessary information to isolate particular values, hence after collection (Stage 1), they undergo parsing (Stage 2).

Log parsing methods such like SPELL [29], DRAIN [13] and IPLom [26, 27] convert each raw log entry e into a structured template. This process identifies a frequently repeated portion of the log, known as log key k_e , and removes the varying parts, defined as event parameters p_e .

Once each entry e is parsed, the sampling phase takes place (Stage 3), with the log keys k_e being sampled into execution sequences S_i based on the nature of the data, each one with a different number N_i of log keys. This is achieved using session window methods, such as those applied to Hadoop Distributed File System (HDFS) logs [42] and OpenStack (OS) logs [6], which can be grouped by *block id* (HDFS) or *instance id* (OS). For logs that do not include session identifiers, like Blue Gene/L (BGL) logs [25], grouping can be done based on timestamps.

Finally, the log embedding phase (Stage 4) concludes the pre-processing workflow. In a first step, the k -th structured sequence of log keys S_i is encoded through well-known Word2Vec model [17], a NN that generates vector representations of words, also known as word embeddings. Word2Vec trains on a large corpus of text, in this case log keys, to capture the contextual relationships between words. It represents these relationships in a continuous vector space where semantically similar words are mapped to nearby points. With this method, each log key k_e is projected into a M -dimensional vector V_e^M , with M sufficiently large to ensure a comprehensive representation of the log key’s semantic information. Thus the resulting processed log sequences are composed into numerical vectors $V_1^M, \dots, V_{N_i}^M$, hence these sequences can be computationally expensive to handle, particularly in tasks involving extensive training data or large vocabularies.

To overcome these computational challenges and enhance efficiency, dimensionality reduction techniques such as the Post-Processing Algorithm (PPA) are commonly employed [35]. PPA’s primary objective is to decrease the dimensionality of Word2Vec embeddings while retaining as much semantic information as possible. This reduction in dimensionality substantially alleviates the computational burden associated with processing and storing the embeddings, making them more suitable for downstream tasks like natural language processing (NLP) and machine learning (ML).

The PPA, proposed in [35], utilizes techniques such as Principal Component Analysis (PCA) to achieve dimensionality reduction. PCA identifies the principal components of the high-dimensional embedding space and projects the embeddings onto a lower-dimensional subspace $n < M$ while preserving as much variance as possible. Additionally, PPA removes d dominating eigenvectors from the dimensionality-reduced embeddings. These dominating eigenvectors, which contribute the most to the variance in the data, are removed to potentially eliminate noise or irrelevant information from the embeddings. The specific number of eigenvectors to remove is typically determined empirically or through cross-validation, depending on the characteristics of the data and the specific application.

3.2 Temporal Convolutional Network architecture for local training

In our work, as also proposed in [6, 45], the training process for embedded data utilizes a Temporal Convolutional Network (TCN), namely a deep NN firstly introduced in [21], which proved to be effective for the log anomaly detection

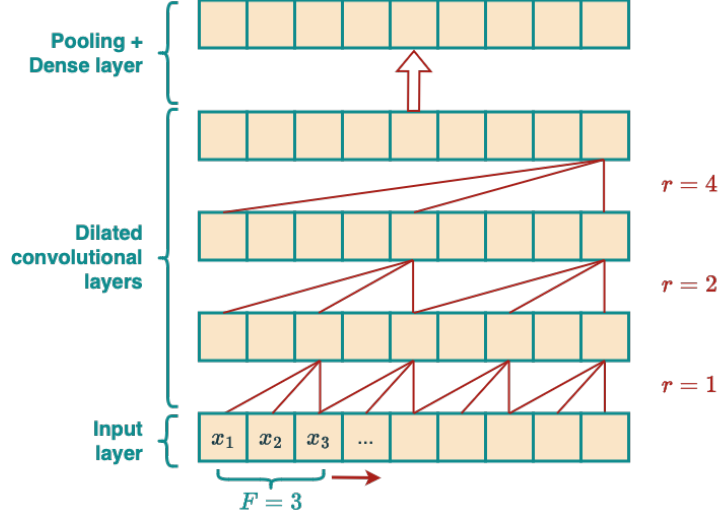


Fig. 2: Typical TCN architecture

problem [45] due to their known capabilities in handling sequential data, that is the type of data inherent to application logs.

Their ability to capture long-term dependencies relies in the usage of *dilated convolutions*, which differs from the causal convolution because of the presence of a dilation rate r . In other words, the latter can be seen as a filter envisaging to skip some input values with a certain step.

From a mathematical perspective, given an input sequence $x = \{x_{[1]}, \dots, x_{[q]}\}$, with $x_{[t]} \in \mathbb{R}^n$ for the generic time $t = 1, \dots, q$, a causal convolution with a filter size F only considers current and past time steps, producing an output

$$y_{[t]} = \sum_{f=0}^{F-1} w_f x_{[t-f]}, \quad (1)$$

with w_f the convolutional filter weights, while a dilated convolution with a dilation rate r modifies the convolution operation (1) to

$$y_{[t]} = \sum_{f=0}^{F-1} w_f x_{[t-rf]}. \quad (2)$$

The output of such dilation convolutional layers, which aim at capturing temporal dependencies at different scales, is handled by global pooling layers which attempt to aggregate temporal features, thus summarizing information from all time steps, across a feature map, which is then processed by fully connected layers, providing the output of the TCN as the binary classification of the log sequence. Fig. 2 shows a typical architecture with three dilated convolutional layers with kernel size $F = 3$.

3.3 Proposed Federated Learning Algorithm

The proposed framework envisages a federation of K local servers in communication with a coordinating one, denoted as G , as shown in Fig. 3. Such a scenario is quite customary in CIs, where ICSs rely on devices capable of storing and processing data, such as local servers, connected to the outside environment via some secure connection. With respect to the coordinating server, it can be either one among the local ones, or may be located in the cloud and can even belong to a different party.

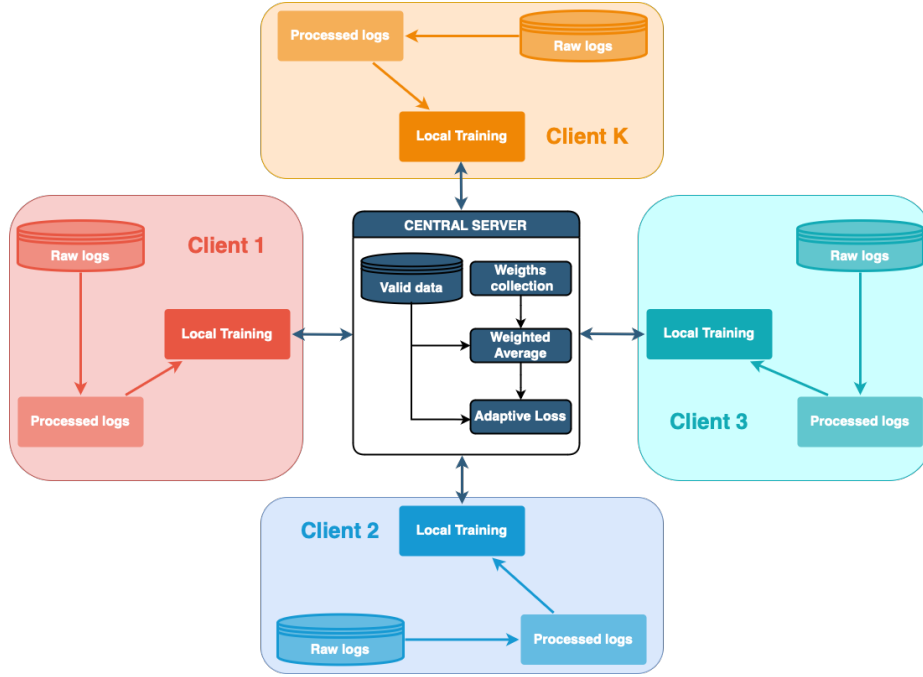


Fig. 3: Log Anomaly Detection FL

The possibility to share the same algorithm with different stakeholders, is made possible by the very nature of FL paradigm, which ensures that no logs nor embedding is shared among the federation. Hence, each local server is in charge of pre-preprocessing data independently, thus exhibiting different embedding, making the proposed scenario close to a real-world application.

Another key difference with respect to other log-related FL implementations [22, 10] lies in the use of an adaptive loss function, resulting from the employment of ADAFED [9]; while, on the one hand, the adaptive nature of the algorithm speeds up convergence, on the other hand allows to evaluate whether or not a client has been subject to a malicious attack, possibly excluding him from the

training procedure, through a performance evaluation mechanism carried out by the coordinator server G .

In particular, the training procedure unfolds as follows: during the c -th *communication round*, $c = 1, \dots, E$, the k -th client, $k = 1, \dots, K$, receives the TCN model from G with current weights W_G^c and trains it over its own local data for T local epochs. Denoting with W_k^c the weights of its model, it sends them over to the coordinator server which evaluates its performance over a dedicated dataset, which we assume to be representative for the machine learning problem at hand, and assigns the corresponding performance weight s_k^c to it. After completing this procedure, G updated the weights of its global model W_G^{c+1} performing the weighted average of all the collected ones

$$W_g^{c+1} = \frac{\sum_{k=1}^K W_k^c s_k^c}{\sum_{k=1}^K s_k^c}. \quad (3)$$

The underlying idea under equation (3) consists in allowing the clients with an higher score to lead the learning procedure, and, consequently, the lesser performing ones to benefit from their experience, regardless of the particular reason for such behaviour; a client may own low quality data or even be subject of a malicious attack. While in the first case it can benefit from the experience of the others, in the second one it can be cut out from the learning procedure with no effect on it.

This performance evaluation strategy also envisages to link the learning of the federation with the global model performance. Due to the distributed nature of data, allowing each client to focus its training over misclassified data proved to boost the performance of shared model [9]; to do so, at the beginning of each communication round, G propagates a weighted loss function along with the newly updated TCN model.

Dealing with a binary classification problem, all the clients attempt to minimize a weighted categorical cross-entropy loss function:

$$\begin{aligned} \ell_k(D_k, c) = & -\frac{1}{|D_k|} \sum_{j=1}^{|D_k|} \left[w_{normal}^c y_j^{D_k} \log(\hat{y}_j^{D_k}) + \right. \\ & \left. + w_{abnormal}^c (1 - y_j^{D_k}) \log(1 - \hat{y}_j^{D_k}) \right], \end{aligned} \quad (4)$$

with D_k the local dataset of the k -th client, $y_j^{D_k}$ and $\hat{y}_j^{D_k}$ its corresponding true and predicted labels, and w_{normal}^c and $w_{abnormal}^c$ representing the above mentioned weights associated to the two classes.

After initially being set to a default value, they are updated at the end of the c -th communication round on the basis of a given performance metric m describing the performance of the global model W_G^{c+1} over the dedicated test set:

$$w_{normal}^{c+1} = \frac{1}{m^{c+1} + \varepsilon}, \quad (5)$$

$$w_{abnormal}^{c+1} = \frac{1}{(1 - m^{c+1}) + \varepsilon}, \quad (6)$$

where $\varepsilon > 0$ is a arbitrarily small value to prevent numerical problems.

The complete algorithm of ADALIGHTLOG FL training is reported in Algorithm 1.

Algorithm 1 ADALIGHTLOG

```

1: for each communication round  $c = 1, \dots, E$  do
2:   for each client  $k = 1, \dots, K$  do
3:     train local servers with adaptive loss weights  $w_{normal}^c$  and  $w_{abnormal}^c$ 
4:     collect model weights  $W_k^c$ 
5:     evaluate and collect the score  $s_k^c$  over  $W_k^c$  on client  $k$  validation set
6:   end for
7:   perform weighted model average  $W_g^{c+1}$  based on the obtained scores as in (3)
8:   compute the metric  $m^{c+1}$  by evaluating  $W_g^{c+1}$  with server  $G$  validation dataset
9:   compute loss function parameters  $w_{normal}^{c+1}$  and  $w_{abnormal}^{c+1}$  as in (5) and (6)
10:  propagate  $W_g^{c+1}$ ,  $w_{normal}^{c+1}$  and  $w_{abnormal}^{c+1}$  to the clients
11: end for

```

4 Simulations

Our numerical simulations are conducted over the HDFS log dataset [42], containing 11.175.629 entries, where approximately 2.58% of them contain anomalies. The chosen parsing method is SPELL [29], which has demonstrated 100% parsing accuracy on the HDFS dataset. The federated framework is depicted in Fig. 3, with the number of local servers fixed at $K = 5$ to preserve parsing accuracy as much as possible. The datasets resulting from a proportional division into K parts consist of approximately $2.235.050 \pm 50$ log entries, providing sufficient data for good parsing accuracy. Each dataset contains 21 ± 2 different log keys.

All the logs for each client are grouped into sequences by their block ID, and sessions are labeled as abnormal (i.e., 1) if there exists at least one log key labeled as abnormal; otherwise, they are labeled as normal (i.e., 0). Clients have approximately 180.000 ± 30.000 normal sessions and 4.500 ± 500 abnormal sessions, with a percentage of abnormal sessions of 2.45%, very close to that of the original database. Log embedding is then performed such that are produced vectors of dimension $M = 300$ as proposed by Wang et al. [45]. By removing $d = 7$ dominating eigenvectors, PPA reduces the embedding dimensionality to $n = N_5 = 19$, as this is the lowest number of log keys among all the local servers

datasets (in particular, in this case in the fifth local server) ; indeed, the number of components used for performing PCA cannot exceed the number of log keys N_i for a given k -th client.

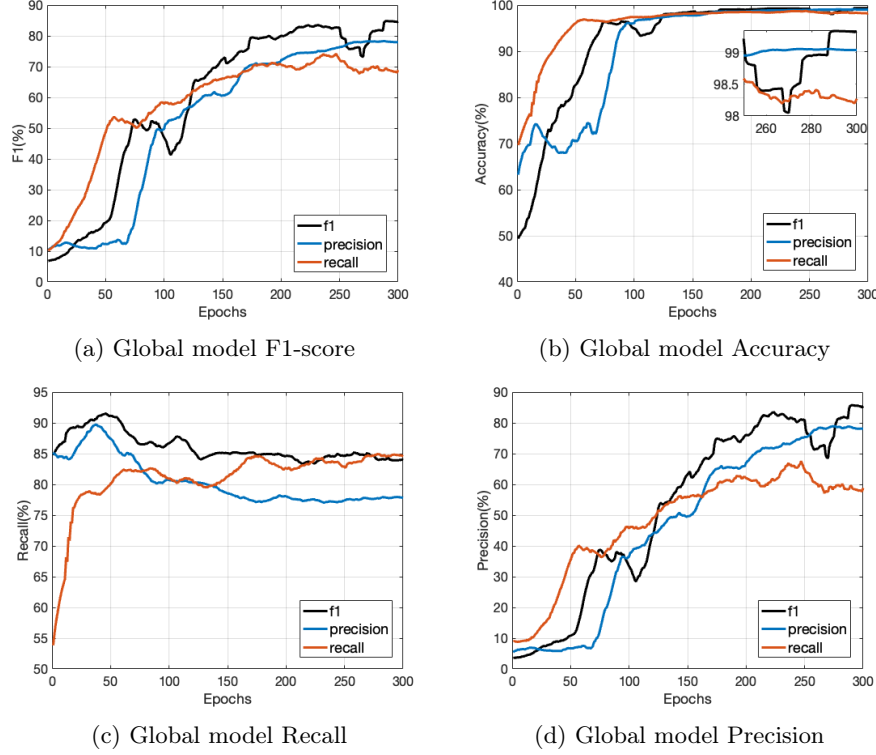


Fig. 4: Comparison over four performance metrics of the m -based algorithm performance: m = F1 black line, m = Precision blue line, m = Recall orange line

For what concerns the simulations, training procedures are performed over $E = 300$ global epochs. During a single communication round each local model is trained for $T = 3$ local epochs, using ADAM optimizer with learning rate $l = 0.005$ and the adaptive loss previously proposed. The local TCNs are composed by four dilated convolutional layers with dilation rates fixed respectively to the values $r = 1, 2, 4, 8$, with an Adaptive Average Pooling layer and a fully-connected linear layer outputting the binary classification of the log sequences.

The first simulation proposes a comparative analysis of the metric m used to evaluate the performance of the global model over the dedicated test set, thus to determine the weights of the adaptive loss function. Evaluating the score s_k of the k -th local client, $k = 1, \dots, K$, as the accuracy over the test set, we compare the influence of the adaptive loss function over the learning procedure when m is set

to be the precision, recall, and F1-score. Even if they are all performance metrics related to classification problems, each one of them focuses on different aspects; while accuracy describes the number of correct predictions over all predictions, precision quantifies how many instances predicted as normal are actually correct and recall measures how many instances are predicted as normal over all normal cases, the F1-score combines the former two as the harmonic mean.

In particular, Fig. 4 shows the results of such comparison, comparing the performance of the the algorithm under a different m over four performance metrics over the dedicated test set, that is F1-score (a), accuracy (b), recall (c), and precision (d).

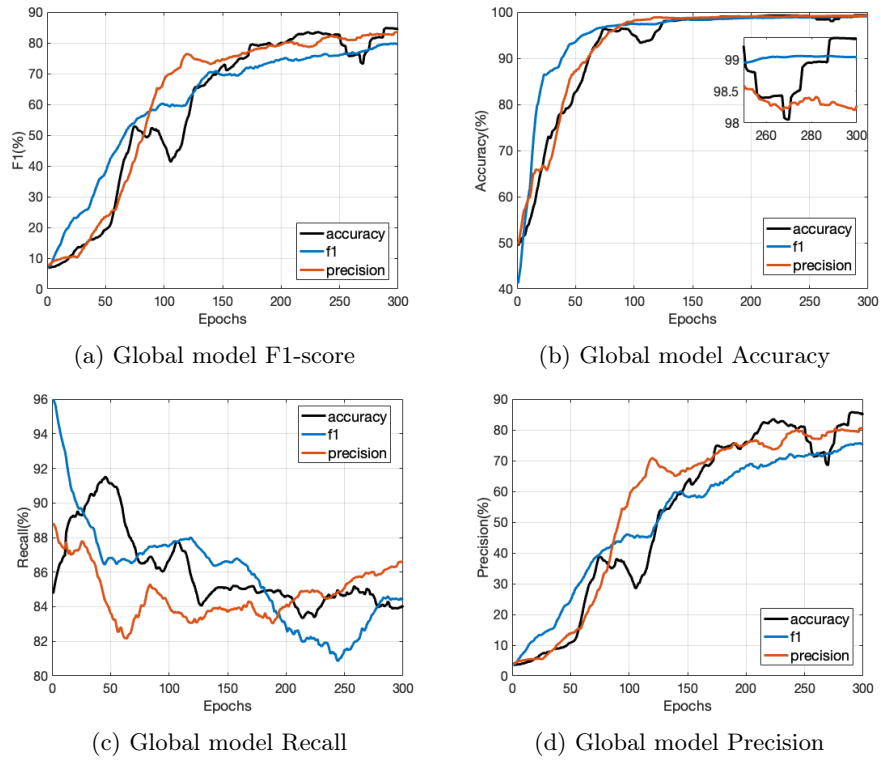


Fig. 5: Comparison over four performance metrics of the s_k -based algorithm performance: s_k = Accuracy black line, s_k = F1 blue line, s_k = Precision orange line

As it is possible to notice from Fig. 4b accuracy reaches values of about 99% in all cases, with the F1 metric-based model slightly outperforming the others. The model using recall metric demonstrates the quickest and most stable convergence across all metrics. It reaches high performance levels early and maintains them

with fewer fluctuations. The F1 metric -based model reaches good accuracy at around $E = 100$, maintaining the highest F1-score and outperforming the other metrics also on the recall.

As the most promising performance metric m seems to be the F1-score, we perform a second comparative analysis related to the local client evaluation score s_k , $k = 1, \dots, K$ among the accuracy, F1-score, and precision. Fixing the global model evaluation metric m to be the F1-score, Fig. 5 shows the results of such comparison, with the accuracy criteria outperforming the others over all the comparative metrics (a)-(b)-(d), except over the recall one (c), where it is outperformed by the F1-score of a 2%.

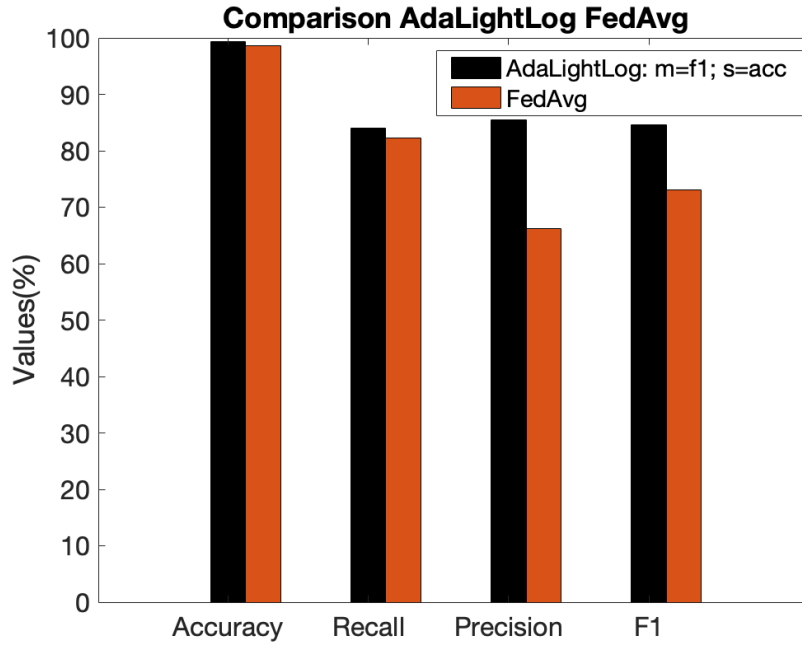


Fig. 6: Performance comparison: ADALIGHTLOG and FEDAVG

Finally, having found that the F1-score is the most promising metrics for m and accuracy for s_k , we compare the resulting proposed algorithm with FEDAVG. The results of such comparison are depicted in Fig. 6, with ADALIGHTLOG outperforming FEDAVG over all the comparative performance metrics.

5 Conclusion and Future Works

This paper has proposed a novel methodology for automatic log anomaly detection that is based on Federated Learning and that makes use of adaptive loss function driven by validation metrics computed by the central server, and dynamic weights for the weighted model averaging based on metrics from its own validation data, in order to achieve the best results, while also protecting the federation from (possibly) tampered models. The proposed approach, namely ADALIGHTLOG, has been showed to outperform traditional Federated Learning approaches like FEDAVG in terms of accuracy, precision and recall. A comparison among the results using different validation metrics both for the adaptive loss and for the dynamic weights has been presented, to achieve the best results.

Future work is aimed at achieving full decentralization of the training procedure leveraging on consensus theory, by also applying finite-time consensus guarantee the convergence of the learning of the agents in a finite-time.

Acknowledgments. This work has been partially supported by the EU in the scope of the NANCY project, which has received funding from the Smart Networks and Services Joint Undertaking (SNS JU) under the European Union’s Horizon Europe research and innovation programme under Grant Agreement No 101096456, and the FSE REACT-EU, PON Ricerca e Innovazione 2014-2020, programme.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Applications of Data Mining in Computer Security. Springer US (2002). <https://doi.org/10.1007/978-1-4615-0953-0>, <http://dx.doi.org/10.1007/978-1-4615-0953-0>
2. Antunes, R.S., André da Costa, C., Küderle, A., Yari, I.A., Eskofier, B.: Federated learning for healthcare: Systematic review and architecture proposal. *ACM Transactions on Intelligent Systems and Technology* **13**(4), 1–23 (May 2022). <https://doi.org/10.1145/3501813>, <http://dx.doi.org/10.1145/3501813>
3. Bonawitz, K., Ivanov, V., Kreuter, B., Marcedone, A., McMahan, H.B., Patel, S., Ramage, D., Segal, A., Seth, K.: Practical secure aggregation for privacy-preserving machine learning. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’17, ACM (Oct 2017). <https://doi.org/10.1145/3133956.3133982>, <http://dx.doi.org/10.1145/3133956.3133982>
4. Botev, A., Lever, G., Barber, D.: Nesterov’s accelerated gradient and momentum as approximations to regularised update descent (2016). <https://doi.org/10.48550/ARXIV.1607.01981>, <https://arxiv.org/abs/1607.01981>
5. Commission, E.: Protecting critical infrastructure in the eu - new rules (2020), https://ec.europa.eu/info/law/better-regulation/have-your-say/initiatives/12462-Protecting-critical-infrastructure-in-the-EU-new-rules_en

6. Du, M., Li, F., Zheng, G., Srikumar, V.: Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. pp. 1285–1298. CCS '17, ACM, New York, NY, USA (2017). <https://doi.org/10.1145/3133956.3134015>, <http://doi.acm.org/10.1145/3133956.3134015>
7. Frankle, J., Carbin, M.: The lottery ticket hypothesis: Finding sparse, trainable neural networks (2018). <https://doi.org/10.48550/ARXIV.1803.03635>, <https://arxiv.org/abs/1803.03635>
8. Geyer, R.C., Klein, T., Nabi, M.: Differentially private federated learning: A client level perspective (2017). <https://doi.org/10.48550/ARXIV.1712.07557>, <https://arxiv.org/abs/1712.07557>
9. Giuseppi, A., Della Torre, L., Menegatti, D., Pietrabissa, A.: Adafed: Performance-based adaptive federated learning. In: 2021 The 5th International Conference on Advances in Artificial Intelligence (ICAAI). ICAAI 2021, ACM (Nov 2021). <https://doi.org/10.1145/3505711.3505717>, <http://dx.doi.org/10.1145/3505711.3505717>
10. Guo, Y., Wu, Y., Zhu, Y., Yang, B., Han, C.: Anomaly detection using distributed log data: A lightweight federated learning approach. In: 2021 International Joint Conference on Neural Networks (IJCNN). IEEE (Jul 2021). <https://doi.org/10.1109/ijcnn52387.2021.9533294>, <http://dx.doi.org/10.1109/IJCNN52387.2021.9533294>
11. Hard, A., Rao, K., Mathews, R., Ramaswamy, S., Beaufays, F., Augenstein, S., Eichner, H., Kiddon, C., Ramage, D.: Federated learning for mobile keyboard prediction (2018). <https://doi.org/10.48550/ARXIV.1811.03604>, <https://arxiv.org/abs/1811.03604>
12. Hawkins, S., He, H., Williams, G., Baxter, R.: Outlier Detection Using Replicator Neural Networks, p. 170–180. Springer Berlin Heidelberg (2002). https://doi.org/10.1007/3-540-46145-0_17, http://dx.doi.org/10.1007/3-540-46145-0_17
13. He, P., Zhu, J., Zheng, Z., Lyu, M.R.: Drain: An online log parsing approach with fixed depth tree. IEEE (2017). <https://doi.org/10.1109/ICWS.2017.13>
14. He, S., Zhu, J., He, P., Lyu, M.R.: Experience report: System log analysis for anomaly detection. In: 2016 IEEE 27th International Symposium on Software Reliability Engineering (ISSRE). IEEE (Oct 2016). <https://doi.org/10.1109/issre.2016.21>, <http://dx.doi.org/10.1109/ISSRE.2016.21>
15. Hu, W., Liao, Y., Vemuri, V.R.: Robust anomaly detection using support vector machines (2003), <https://api.semanticscholar.org/CorpusID:2433033>
16. Jithish, J., Alangot, B., Mahalingam, N., Yeo, K.S.: Distributed anomaly detection in smart grids: A federated learning-based approach. IEEE Access **11**, 7157–7179 (2023). <https://doi.org/10.1109/access.2023.3237554>, <http://dx.doi.org/10.1109/ACCESS.2023.3237554>
17. Joulin, A., Grave, E., Bojanowski, P., Douze, M., Jégou, H., Mikolov, T.: Fast-text.zip: Compressing text classification models. arXiv preprint arXiv:1612.03651 (2016), <https://arxiv.org/abs/1612.03651>
18. Kruegel, C., Mutz, D., Robertson, W., Valeur, F.: Bayesian event classification for intrusion detection. In: 19th Annual Computer Security Applications Conference, 2003. Proceedings. IEEE. <https://doi.org/10.1109/csac.2003.1254306>, <http://dx.doi.org/10.1109/CSAC.2003.1254306>
19. Krügel, C., Toth, T., Kirda, E.: Service specific anomaly detection for network intrusion detection. In: Proceedings of the 2002 ACM symposium on Applied

- computing. SAC02, ACM (Mar 2002). <https://doi.org/10.1145/508791.508835>, <http://dx.doi.org/10.1145/508791.508835>
20. Kumar, M.R., Devi, B.R., Rangaswamy, K., Sangeetha, M., Kumar, K.V.R.: Iot-edge computing for efficient and effective information process on industrial automation. In: 2023 International Conference on Networking and Communications (ICNWC). IEEE (Apr 2023). <https://doi.org/10.1109/icnwc57852.2023.10127492>, <http://dx.doi.org/10.1109/ICNWC57852.2023.10127492>
 21. Lea, C., Vidal, R., Reiter, A., Hager, G.D.: Temporal Convolutional Networks: A Unified Approach to Action Segmentation, p. 47–54. Springer International Publishing (2016). https://doi.org/10.1007/978-3-319-49409-8_7, http://dx.doi.org/10.1007/978-3-319-49409-8_7
 22. Li, B., Ma, S., Deng, R., Choo, K.K.R., Yang, J.: Federated anomaly detection on system logs for the internet of things: A customizable and communication-efficient approach. *IEEE Transactions on Network and Service Management* **19**(2), 1705–1716 (Jun 2022). <https://doi.org/10.1109/tnsm.2022.3152620>, <http://dx.doi.org/10.1109/TNSM.2022.3152620>
 23. Li, T., Sahu, A.K., Zaheer, M., Sanjabi, M., Talwalkar, A., Smith, V.: Federated optimization in heterogeneous networks (2018). <https://doi.org/10.48550/ARXIV.1812.06127>, <https://arxiv.org/abs/1812.06127>
 24. Li, X., Huang, K., Yang, W., Wang, S., Zhang, Z.: On the convergence of fedavg on non-iid data (2019). <https://doi.org/10.48550/ARXIV.1907.02189>, <https://arxiv.org/abs/1907.02189>
 25. Liu, X., Liu, W., Di, X., Li, J., Cai, B., Ren, W., Yang, H.: Lognads: Network anomaly detection scheme based on semantic representation. *Future Generation Computer Systems* **124**, 390–405 (2021). <https://doi.org/10.1016/j.future.2021.04.048>
 26. Makanju, A., Zincir-Heywood, A.N., Milios, E.E.: Clustering event logs using iterative partitioning. In: Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 1255–1264. KDD '09, ACM, New York, NY, USA (2009). <https://doi.org/10.1145/1557019.1557152>
 27. Makanju, A., Zincir-Heywood, A.N., Milios, E.E.: A lightweight algorithm for message type extraction in system application logs. *IEEE Transactions on Knowledge and Data Engineering* **24**(11), 1921–1936 (2012)
 28. McMahan, H.B., Moore, E., Ramage, D., Hampson, S., Arcas, B.A.y.: Communication-efficient learning of deep networks from decentralized data (2016). <https://doi.org/10.48550/ARXIV.1602.05629>, <https://arxiv.org/abs/1602.05629>
 29. Min, D., Feifei, L.: Spell: Streaming parsing of system event logs. *IEEE* (2016). <https://doi.org/10.1109/ICDM.2016.160>
 30. Nardi, M., Valerio, L., Passarella, A.: Anomaly detection through unsupervised federated learning (2022). <https://doi.org/10.48550/ARXIV.2209.04184>, <https://arxiv.org/abs/2209.04184>
 31. Nguyen, D.C., Ding, M., Pathirana, P.N., Seneviratne, A., Li, J., Vincent Poor, H.: Federated learning for internet of things: A comprehensive survey. *IEEE Communications Surveys; Tutorials* **23**(3), 1622–1658 (2021). <https://doi.org/10.1109/comst.2021.3075439>, <http://dx.doi.org/10.1109/COMST.2021.3075439>
 32. Poojitha, G., Kumar, K.N., Reddy, P.J.: Intrusion detection using artificial neural network. In: 2010 Second International conference on Computing, Communication and Networking Technologies. *IEEE* (Jul 2010). <https://doi.org/10.1109/icccnt.2010.5592568>, <http://dx.doi.org/10.1109/ICCCNT.2010.5592568>

33. Preuveneers, D., Rimmer, V., Tsingenopoulos, I., Spooren, J., Joosen, W., Ilie-Zudor, E.: Chained anomaly detection models for federated learning: An intrusion detection case study. *Applied Sciences* **8**(12), 2663 (Dec 2018). <https://doi.org/10.3390/app8122663>, <http://dx.doi.org/10.3390/app8122663>
34. Qu, Z., Lin, K., Li, Z., Zhou, J., Zhou, Z.: A unified linear speedup analysis of federated averaging and nesterov fedavg (2020). <https://doi.org/10.48550/ARXIV.2007.05690>, <https://arxiv.org/abs/2007.05690>
35. Raunak, V.: Effective dimensionality reduction for word embeddings. In: *Proceedings of the Workshop on Representation Learning for NLP (RepL4NLP) at ACL* (2019)
36. Selim, G.E.I., Hemdan, E.E.D., Shehata, A.M., El-Fishawy, N.A.: Anomaly events classification and detection system in critical industrial internet of things infrastructure using machine learning algorithms. *Multimedia Tools and Applications* **80**(8), 12619–12640 (Jan 2021). <https://doi.org/10.1007/s11042-020-10354-1>, <http://dx.doi.org/10.1007/s11042-020-10354-1>
37. Trivedi, S., Anh Tran, T., Faruqi, N., Hassan, M.M.: An exploratory analysis of effect of adversarial machine learning attack on iot-enabled industrial control systems. In: *2023 International Conference on Smart Computing and Application (ICSCA)*. IEEE (Feb 2023). <https://doi.org/10.1109/icsca57840.2023.10087713>, <http://dx.doi.org/10.1109/ICSCA57840.2023.10087713>
38. Vucovich, M., Tarcar, A., Rebelo, P., Gade, N., Porwal, R., Rahman, A., Redino, C., Choi, K., Nandakumar, D., Schiller, R., Bowen, E., West, A., Bhat-tacharya, S., Veeramani, B.: Anomaly detection via federated learning (2022). <https://doi.org/10.48550/ARXIV.2210.06614>, <https://arxiv.org/abs/2210.06614>
39. Wang, H.P., Stich, S.U., He, Y., Fritz, M.: ProgFed: Effective, communication, and computation efficient federated learning by progressive training (2021). <https://doi.org/10.48550/ARXIV.2110.05323>, <https://arxiv.org/abs/2110.05323>
40. Wang, X., Liu, W., Lin, H., Hu, J., Kaur, K., Hossain, M.S.: Ai-empowered trajectory anomaly detection for intelligent transportation systems: A hierarchical federated learning approach. *IEEE Transactions on Intelligent Transportation Systems* **24**(4), 4631–4640 (Apr 2023). <https://doi.org/10.1109/tits.2022.3209903>, <http://dx.doi.org/10.1109/TITS.2022.3209903>
41. Wu, C., Wu, F., Lyu, L., Huang, Y., Xie, X.: Communication-efficient federated learning via knowledge distillation. *Nature Communications* **13**(1) (Apr 2022). <https://doi.org/10.1038/s41467-022-29763-x>, <http://dx.doi.org/10.1038/s41467-022-29763-x>
42. Xu, W., Huang, L., Fox, A., Patterson, D., Jordan, M.I.: Detecting large-scale system problems by mining console logs. In: *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles (SOSP '09)*. pp. 117–132. ACM, New York, NY, USA (October 2009)
43. Ye, N., Chen, Q.: An anomaly detection technique based on a chi-square statistic for detecting intrusions into information systems. *Quality and Reliability Engineering International* **17**(2), 105–112 (Mar 2001). <https://doi.org/10.1002/qre.392>, <http://dx.doi.org/10.1002/qre.392>
44. Zhang, Y., Duan, L., Cheung, N.M.: Accelerating federated learning on non-iid data against stragglers. In: *2022 IEEE International Conference on Sensing, Communication, and Networking (SECON Workshops)*. IEEE (Sep 2022). <https://doi.org/10.1109/seconworkshops56311.2022.9926402>, <http://dx.doi.org/10.1109/SECONWorkshops56311.2022.9926402>

45. Zumin, W., Jiyu, T., Hui, F., Liming, C., Jing, Q.: Lightlog: A lightweight temporal convolutional network for log anomaly detection on the edge. Elsevier **203** (2022). <https://doi.org/https://doi.org/10.1016/j.comnet.2021.108616>